Black-box tests are designed to validate functional requirements without regard to the internal workings of a program. Black-box testing techniques focus on the information domain of the software, deriving test cases by partitioning the input and output domain of a program in a manner that provides thorough test coverage. Equivalence partitioning divides the input domain into classes of data that are likely to exercise a specific software function. Boundary value analysis probes the program's ability to handle data at the limits of acceptability.

Unlike testing (a systematic, planned activity), debugging can be viewed as an art. Beginning with a symptomatic indication of a problem, the debugging activity must track down the cause of an error. Testing can sometimes help find the root cause of the error. But often, the most valuable resource is the counsel of other members of the software engineering staff.

PROBLEMS AND POINTS TO PONDER

19.1. Using your own words, describe the difference between verification and validation. Do both make use of test-case design methods and testing strategies?

19.2. List some problems that might be associated with the creation of an independent test group. Are an ITG and an SQA group made up of the same people?

19.3. Why is a highly coupled module difficult to unit test?

19.4. Is unit testing possible or even desirable in all circumstances? Provide examples to justify your answer.

19.5. Can you think of any additional testing objectives that are not discussed in Section 19.1.1?

19.6. Select a software component that you have designed and implemented recently. Design a set of test cases that will ensure that all statements have been executed using basis path testing.

19.7. Myers [Mye79] uses the following program as a self-assessment for your ability to specify adequate testing: A program reads three integer values. The three values are interpreted as representing the lengths of the sides of a triangle. The program prints a message that states whether the triangle is scalene, isosceles, or equilateral. Develop a set of test cases that you feel will adequately test this program.

19.8. Design and implement the program (with error handling where appropriate) specified in Problem 19.7. Derive a flow graph for the program and apply basis path testing to develop test cases that will guarantee that all statements in the program have been tested. Execute the cases and show your results.

19.9. Give at least three examples in which black-box testing might give the impression that "everything's OK," while white-box tests might uncover an error. Give at least three examples in which white-box testing might give the impression that "everything's OK," while black-box tests might uncover an error.

19.10. In your own words, describe why the class is the smallest reasonable unit for testing within an OO system.

Design element: Quick Look icon magnifying glass: © Roger Pressman

CHAPTER

20 SOFTWARE TESTING—INTEGRATION LEVEL

A single developer may be able to test software components without involving other team members. This is not true for integration testing where a component must interact properly with components developed by other team members. Integration testing exposes many weaknesses of software development groups who have not gelled as a team. Integration testing presents an interesting dilemma for software engineers, who by their nature are constructive people. In fact, all testing requires that developers discard preconceived notions of the "correctness" of software just developed and instead work hard to design test cases to "break" the software. This means that team members need to be able to accept suggestions from other team members that their code is not behaving properly when it is tested as part of the latest software increment.

KEY CONCEPTS

artificial intelligence 403
black-box testing 397
bottom-up integration 399
continuous integration 400
cluster testing 404
fault-based testing 405
integration testing 398
multiple-class partition testing 405
patterns 409
regression testing 402
scenario-based testing 407
smoke testing 400
thread-based testing 404
top-down integration 398
validation testing 407
white-box testing 397



QUICK LOOK

What is it? Integration testing assembles components in a manner that allows the testing of increasingly larger software functions with the intent of finding errors as the software is assembled.

Who does it? During early stages of testing, a software engineer performs all tests. However, as the testing process progresses, testing specialists may become involved in addition to other stakeholders.

Why is it important? Test cases must be designed using disciplined techniques to ensure that the components have been integrated properly into the complete software product.

What are the steps? Internal program logic is exercised using "white-box" test-case design techniques and software requirements are exercised using "black-box" test-case design techniques.

What is the work product? A set of test cases designed to exercise internal logic, interfaces, component collaborations, and external requirements is designed and documented, expected results are defined, and actual results are recorded.

How do I ensure that I've done it right? When you begin testing, change your point of view. Try hard to "break" the software! Design test cases in a disciplined fashion, and review the test cases you do create for thoroughness.

Beizer [Bei90] describes a "software myth" that all testers face. He writes: "There's a myth that if we were really good at programming, there would be no bugs to catch . . . There are bugs, the myth says, because we are bad at what we do; and if we are bad at it, we should feel guilty about it."

Should testing instill guilt? Is testing really destructive? The answer to these questions is, No!

At the beginning of this book, we stressed the fact that software is only one element of a larger computer-based system. Ultimately, software is incorporated with other system elements (e.g., hardware, people, information), and *systems testing* (a series of system integration and validation tests) is conducted. These tests fall outside the scope of the software process and are not conducted solely by software engineers. However, steps taken during software design and testing can greatly improve the probability of successful software integration in the larger system.

In this chapter, we discuss techniques for software integration testing strategies applicable to most software applications. Specialized software testing strategies are discussed in Chapter 21.

20.1 SOFTWARE TESTING FUNDAMENTALS

The goal of testing is to find errors, and a good test is one that has a high probability of finding an error. Kaner, Falk, and Nguyen [Kan93] suggest the following attributes of a "good" test:

A good test has a high probability of finding an error. To achieve this goal, the tester must understand the software and attempt to develop a mental picture of how the software might fail.

A good test is not redundant. Testing time and resources are limited. There is no point in conducting a test that has the same purpose as another test. Every test should have a different purpose (even if it is subtly different).

A good test should be "best of breed" [Kan93]. In a group of tests that have a similar intent, time and resource limitations may dictate the execution of only those tests that have the highest likelihood of uncovering a whole class of errors.

A good test should be neither too simple nor too complex. Although it is sometimes possible to combine a series of tests into one test case, the possible side effects associated with this approach may mask errors. In general, each test should be executed separately.

Any engineered product (and most other things) can be tested in one of two ways: (1) Knowing the specified function that a product has been designed to perform, tests can be conducted that demonstrate each function is fully operational while at the same time searching for errors in each function. (2) Knowing the internal workings of a product, tests can be conducted to ensure that "all gears mesh," that is, internal operations are performed according to specifications and all internal components have been adequately exercised. The first test approach

takes an external view of testing and is called *black-box testing*. The second requires an internal view of testing and is termed *white-box testing*.[1] Both are useful in integration testing [Jan16].

20.1.1 Black-Box Testing

Black-box testing alludes to integration testing that is conducted by exercising the component interfaces with other components and with other systems. It examines some fundamental aspect of a system with little regard for the internal logical structure of the software. Instead, the focus is on ensuring the component executes correctly in the larger software build when the input data and software context specified by its preconditions is correct and behaves in the ways specified by its postconditions. It is of course important to make sure that the component behaves correctly when its preconditions are not satisfied (e.g., it can handle bad inputs without crashing).

Black-box testing is based on the requirements specified in user stories (Chapter 7). Test-case authors do not need to wait for the component implementation code to be written once the component interface is defined. Several cooperating components may need to be written to implement the functionality defined by a single user story. Validation testing (Section 20.5) often defines black-box test cases in terms of the end-user visible input actions and observable output behaviors, without any knowledge of how the components themselves were implemented.

20.1.2 White-Box Testing

White-box testing, sometimes called *glass-box testing* or *structural testing,* is an integration testing philosophy that uses implementation knowledge of the control structures described as part of component-level design to derive test cases. White-box testing of software is predicated on close examination of procedural implementation details and data structure implementation details. White-box tests can be designed only after component-level design (or source code) exists. The logical details of the program must be available. Logical paths through the software and collaborations between components are the focus of white-box integration testing.

At first glance it would seem that very thorough white-box testing would lead to "100 percent correct programs." All we need do is define all logical paths, develop test cases to exercise them, and evaluate results, that is, generate test cases to exercise program logic exhaustively. Unfortunately, exhaustive testing presents certain logistical problems. For even small programs, the number of possible logical paths can be very large. White-box testing should not, however, be dismissed as impractical. Testers should select a reasonable number of important logical paths to exercise once component integration occurs. Important data structures should also be tested for validity after component integration.

1 The terms *functional testing* and *structural testing* are sometimes used in place of black-box and white-box testing, respectively.

20.2 INTEGRATION TESTING

A neophyte in the software world might ask a seemingly legitimate question once all modules have been unit-tested: "If they all work individually, why do you doubt that they'll work when we put them together?" The problem, of course, is "putting them together"—interfacing. Data can be lost across an interface; one component can have an inadvertent, adverse effect on another; subfunctions, when combined, may not produce the desired major function; individually acceptable imprecision may be magnified to unacceptable levels; and global data structures can present problems. Sadly, the list goes on and on.

Integration testing is a systematic technique for constructing the software architecture while at the same time conducting tests to uncover errors associated with interfacing. The objective is to take unit-tested components and build a program structure that has been dictated by design.

There is often a tendency to attempt nonincremental integration, that is, to construct the program using a "big bang" approach. In the big bang approach, all components are combined in advance and the entire program is tested as a whole. Chaos usually results! Errors are encountered, but correction is difficult because isolation of causes is complicated by the vast expanse of the entire program. Taking the big bang approach to integration is a lazy strategy that is doomed to failure.

Incremental integration is the antithesis of the big bang approach. The program is constructed and tested in small increments, where errors are easier to isolate and correct; interfaces are more likely to be tested completely; and a systematic test approach may be applied. Integrate incrementally and testing as you go is a more cost-effective strategy. We discuss several common incremental integration testing strategies in the remainder of this chapter.

20.2.1 Top-Down Integration

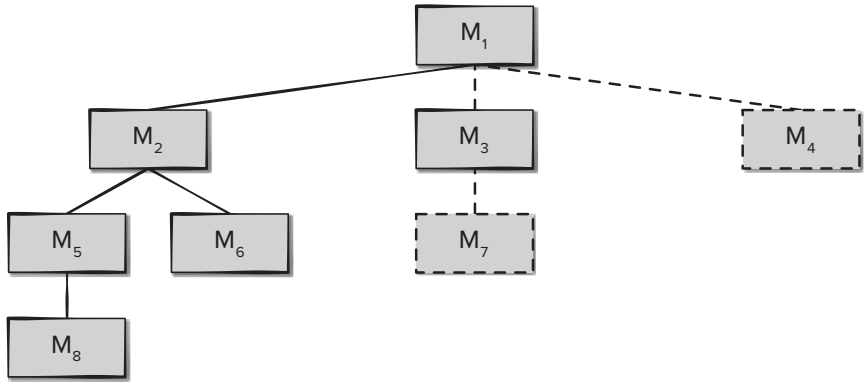
Top-down integration testing is an incremental approach to construction of the software architecture. Modules (also referred to as components in this book) are integrated by moving downward through the control hierarchy, beginning with the main control module (main program). Modules subordinate (and ultimately subordinate) to the main control module are incorporated into the structure in either a depth-first or breadth-first manner.

Referring to Figure 20.1, *depth-first integration* integrates all components on a major control path of the program structure. Selection of a major path is somewhat arbitrary and depends on application-specific characteristics (e.g., components needed to implement one use case). For example, selecting the left-hand path, components M_1, M_2, M_5 would be integrated first. Next, M_8 or (if necessary for proper functioning of M_2) M_6 would be integrated. Then, the central and right-hand control paths are built. *Breadth-first integration* incorporates all components directly subordinate at each level, moving across the structure horizontally. From the figure, components M_2, M_3, and M_4 would be integrated first. The next control level, M_5, M_6, and so on, follows. The integration process is performed in a series of five steps:

1. The main control module is used as a test driver, and stubs are substituted for all components directly subordinate to the main control module.

FIGURE 20.1

Top-down integration



2. Depending on the integration approach selected (i.e., depth or breadth first), subordinate stubs are replaced one at a time with actual components.
3. Tests are conducted as each component is integrated.
4. On completion of each set of tests, another stub is replaced with the real component.
5. Regression testing (discussed later in this section) may be conducted to ensure that new errors have not been introduced.

The process continues from step 2 until the entire program structure is built.

The top-down integration strategy verifies major control or decision points early in the test process. In a "well-factored" program structure, decision making occurs at upper levels in the hierarchy and is therefore encountered first. If major control problems do exist, early recognition is essential. If depth-first integration is selected, a complete function of the software may be implemented and demonstrated. Early demonstration of functional capability is a confidence builder for all stakeholders.

20.2.2 Bottom-Up Integration

Bottom-up integration testing, as its name implies, begins construction and testing with *atomic modules* (i.e., components at the lowest levels in the program structure). Bottom-up integration eliminates the need for complex stubs. Because components are integrated from the bottom up, the functionality provided by components subordinate to a given level is always available and the need for stubs is eliminated. A bottom-up integration strategy may be implemented with the following steps:

1. Low-level components are combined into clusters (sometimes called *builds*) that perform a specific software subfunction.
2. A *driver* (a control program for testing) is written to coordinate test-case input and output.
3. The cluster is tested.
4. Drivers are removed and clusters are combined, moving upward in the program structure.